

AGH Racing DV Simulator

Model Equations

AGH Racing

Contents

1	Scope and notation	1
2	Planar four-wheel vehicle dynamics	2
3	Control modes and torque routing	2
3.1	ONE_WHEEL	2
3.2	FOUR_WHEEL	3
4	MF6.1 tyre model	3
4.1	Slip variables	3
4.2	Pure longitudinal and lateral force	3
4.3	Combined slip and force relaxation	4
5	Suspension and normal loads	4
6	Steering and drivetrain	5
7	Camera model	5
8	Lidar model	5
9	Fusion and perception errors	6
10	IMU	6
11	State estimator	7
12	Discrete timing	7
13	Implementation references	7

1 Scope and notation

This document is the mathematical reference for the models implemented by the simulator. The C++ implementation remains the executable definition. All quantities use SI units. Wheel indices are

$$i \in \{\text{FL}, \text{FR}, \text{RL}, \text{RR}\}.$$

The vehicle body frame uses longitudinal velocity v_x , lateral velocity v_y , yaw angle ψ , and yaw rate $r = \dot{\psi}$.

2 Planar four-wheel vehicle dynamics

Tyre forces are calculated in each wheel frame and rotated into the body frame. The body equations are

$$\dot{v}_x = \frac{\sum_i F_{x,i}^b - F_{\text{res}}}{m} + v_y r, \quad (1)$$

$$\dot{v}_y = \frac{\sum_i F_{y,i}^b}{m} - v_x r, \quad (2)$$

$$\dot{r} = \frac{M_z}{I_z}. \quad (3)$$

The yaw moment follows directly from the wheel positions

$$M_z = \sum_i \left(x_i F_{y,i}^b - y_i F_{x,i}^b \right). \quad (4)$$

Global pose kinematics are

$$\dot{x} = v_x \cos \psi - v_y \sin \psi, \quad (5)$$

$$\dot{y} = v_x \sin \psi + v_y \cos \psi, \quad (6)$$

$$\dot{\psi} = r. \quad (7)$$

The longitudinal resistance model is

$$F_{\text{roll}} = C_r m g, \quad (8)$$

$$F_{\text{drag}} = C_d |v_x|^2, \quad (9)$$

$$F_{\text{extra}} = F_0 + k_v |v_x|, \quad (10)$$

$$F_{\text{res}} = \text{sgn}(v_x) (F_{\text{roll}} + F_{\text{drag}} + F_{\text{extra}}). \quad (11)$$

Aerodynamic downforce is split between axles:

$$F_{\text{aero},F} = C_{l,F} v_x^2, \quad F_{\text{aero},R} = C_{l,R} v_x^2. \quad (12)$$

Each wheel has an independent angular-velocity state

$$I_w \dot{\omega}_i = T_i - R F_{x,i}. \quad (13)$$

3 Control modes and torque routing

The two values of `dv_interfaces/Control.move_type` use separate paths.

3.1 ONE_WHEEL

Only this mode uses baseline allocation and torque vectoring. The total motor torque request is

$$T_{\Sigma}^m = T_{\text{FL}}^m + T_{\text{FR}}^m + T_{\text{RL}}^m + T_{\text{RR}}^m. \quad (14)$$

After the fixed drivetrain ratio i_g , baseline wheel torques are

$$T_{\text{FL}}^0 = T_{\text{FR}}^0 = \frac{1}{2} \lambda_F i_g T_{\Sigma}^m, \quad (15)$$

$$T_{\text{RL}}^0 = T_{\text{RR}}^0 = \frac{1}{2} (1 - \lambda_F) i_g T_{\Sigma}^m, \quad (16)$$

where λ_F has separate configured values for drive and braking. Torque vectoring applies a bounded symmetric correction

$$\begin{bmatrix} T_{\text{FL}} \\ T_{\text{FR}} \\ T_{\text{RL}} \\ T_{\text{RR}} \end{bmatrix} = \begin{bmatrix} T_{\text{FL}}^0 \\ T_{\text{FR}}^0 \\ T_{\text{RL}}^0 \\ T_{\text{RR}}^0 \end{bmatrix} + \begin{bmatrix} +\Delta T \\ -\Delta T \\ +\Delta T \\ -\Delta T \end{bmatrix}. \quad (17)$$

3.2 FOUR_WHEEL

Baseline allocation and torque vectoring are bypassed. Each request is converted independently:

$$T_i = i_g T_i^m. \quad (18)$$

4 MF6.1 tyre model

4.1 Slip variables

For local wheel-frame velocities $v_{x,i}$, $v_{y,i}$,

$$v_{\text{den}}(v_x) = \begin{cases} |v_x|, & |v_x| > v_{\text{th}}, \\ \frac{1}{2} \left(v_{\text{th}} + \frac{v_x^2}{v_{\text{th}}} \right), & |v_x| \leq v_{\text{th}}, \end{cases} \quad (19)$$

$$\kappa_i = \text{clip} \left(\frac{\omega_i R - v_{x,i}}{v_{\text{den}}(v_{x,i})}, -0.99, 0.99 \right), \quad (20)$$

$$\alpha_i = -\text{atan2}(v_{y,i}, v_{x,i,\text{safe}}). \quad (21)$$

The implementation uses $v_{\text{th}} = 1.5 \text{ m s}^{-1}$.

4.2 Pure longitudinal and lateral force

For either longitudinal slip or slip angle, the common Magic Formula form is

$$\phi = Bx - E (Bx - \arctan(Bx)), \quad (22)$$

$$F(x) = D \sin(C \arctan \phi). \quad (23)$$

Load sensitivity uses

$$df_z = \frac{F_z - F_{z0}}{F_{z0}}, \quad (24)$$

with peak factors

$$\mu_x = \lambda_x(p_{Dx1} + p_{Dx2}df_z), \quad \mu_y = \lambda_y(p_{Dy1} + p_{Dy2}df_z), \quad (25)$$

and

$$D_x = \mu_x F_z, \quad D_y = \mu_y F_z. \quad (26)$$

The longitudinal stiffness term is

$$K_{x\kappa} = F_z(p_{Kx1} + p_{Kx2}df_z) \exp(p_{Kx3}df_z), \quad B_x = \frac{K_{x\kappa}}{C_x D_x}. \quad (27)$$

The lateral stiffness term is

$$K_{y\alpha} = p_{Ky1} F_{z0} \sin \left(p_{Ky4} \arctan \frac{F_z}{p_{Ky2} F_{z0}} \right), \quad B_y = \frac{K_{y\alpha}}{C_y D_y}. \quad (28)$$

Horizontal shifts are applied before evaluating the curves, and the force value at zero physical slip is subtracted to remove the shift-induced offset.

4.3 Combined slip and force relaxation

Pure forces are limited by a friction ellipse. Define

$$s = \sqrt{\left(\frac{F_{x0}}{\mu_x F_z}\right)^2 + \left(\frac{F_{y0}}{\mu_y F_z}\right)^2}. \quad (29)$$

Then

$$(F_x, F_y) = \begin{cases} (F_{x0}, F_{y0}), & s \leq 1, \\ (F_{x0}/s, F_{y0}/s), & s > 1. \end{cases} \quad (30)$$

With $V = \sqrt{v_x^2 + v_{th}^2}$, the dynamic relaxation lengths are

$$L_\alpha = \frac{V d_y}{c_y} + \frac{1}{c_y} \left| \frac{\partial F_y}{\partial \alpha} \right|, \quad (31)$$

$$L_\kappa = \frac{V d_x}{c_x} + \frac{1}{c_x} \left| \frac{\partial F_x}{\partial \kappa} \right|. \quad (32)$$

For a frozen target over one integration substep,

$$F^+ = F + (F_{MF} - F) \left(1 - \exp \left[-\frac{V \Delta t}{L} \right] \right). \quad (33)$$

5 Suspension and normal loads

Each wheel normal load is decomposed as

$$N_i = N_{\text{direct},i} + N_{\text{lat,el},i} + N_{\text{long,el},i}. \quad (34)$$

The direct component contains static load, aerodynamic downforce, unsprung transfer, and geometric transfer.

For $L = l_f + l_r$, static axle loads are

$$F_{z,F}^0 = \frac{m g l_r}{L}, \quad F_{z,R}^0 = \frac{m g l_f}{L}. \quad (35)$$

Longitudinal unsprung and geometric transfer are

$$\Delta F_{z,\text{long},u} = \frac{a_x (m_{uF} h_{uF} + m_{uR} h_{uR})}{L}, \quad (36)$$

$$\Delta F_{z,\text{long},g} = k_{gF} F_{x,F,s} + k_{gR} F_{x,R,s}. \quad (37)$$

The elastic longitudinal target is

$$\Delta F_{z,\text{long},el}^* = \frac{m_s a_x h_s}{L} - \Delta F_{z,\text{long},g}. \quad (38)$$

Per-axle lateral direct transfer is

$$\Delta F_{z,u,j} = \frac{m_{u,j} a_y h_{u,j}}{t_j}, \quad (39)$$

$$\Delta F_{z,g,j} = \frac{F_{y,j,s} h_{RC,j}}{t_j}, \quad j \in \{F, R\}. \quad (40)$$

The remaining sprung roll moment is

$$M_{\text{roll},el}^* = m_s a_y h_s - (F_{y,F,s} h_{RC,F} + F_{y,R,s} h_{RC,R}). \quad (41)$$

It is split between axles:

$$\Delta F_{z,el,F}^* = \frac{\lambda_\phi M_{\text{roll},el}^*}{t_F}, \quad \Delta F_{z,el,R}^* = \frac{(1 - \lambda_\phi) M_{\text{roll},el}^*}{t_R}. \quad (42)$$

Elastic pitch and roll loads use independent second-order dynamics:

$$\ddot{N}_{\text{el}} = \omega^2 (N_{\text{el}}^* - N_{\text{el}}) - 2\zeta\omega \dot{N}_{\text{el}}. \quad (43)$$

The previous-step acceleration is used to avoid the algebraic loop $N \rightarrow F_{\text{tyre}} \rightarrow a \rightarrow N$.

6 Steering and drivetrain

The steering rack is a second-order system:

$$\ddot{\delta}_r = \omega_n^2 (\delta_{\text{request}} - \delta_r) - 2\zeta\omega_n \dot{\delta}_r. \quad (44)$$

Left and right wheel angles are obtained from the rack geometry. The command and measured encoder include independent configured noise terms; the actuator command additionally includes steering bias.

Each drivetrain torque state follows

$$\dot{T}_i = \frac{T_{i,\text{request}} - T_i}{\tau_d}. \quad (45)$$

Per-wheel torque is limited by the configured torque bounds and by one quarter of the total drive or regenerative power limit.

7 Camera model

For a cone at global coordinates (x_c, y_c, z_c) , the planar transformation to the vehicle frame is

$$\begin{bmatrix} x_b \\ y_b \end{bmatrix} = \begin{bmatrix} \cos \psi & \sin \psi \\ -\sin \psi & \cos \psi \end{bmatrix} \begin{bmatrix} x_c - x \\ y_c - y \end{bmatrix}. \quad (46)$$

Sensor offsets are added to obtain (x_s, y_s, z_s) . A cone is visible when

$$x_s > 0, \quad d \leq d_{\text{max}}, \quad |y_s| \leq x_s \tan \frac{FOV_h}{2}, \quad |z_s| \leq x_s \tan \frac{FOV_v}{2}. \quad (47)$$

Position noise grows exponentially with distance:

$$RMSE(d) = a \exp(bd), \quad (48)$$

$$\sigma_{xy} = \frac{RMSE(d)}{\sqrt{2}}, \quad (49)$$

$$x_{\text{obs}} = x_s + \sigma_{xy} \eta_x, \quad y_{\text{obs}} = y_s + \sigma_{xy} \eta_y, \quad (50)$$

where $\eta_x, \eta_y \sim \mathcal{N}(0, 1)$.

8 Lidar model

After the body transform and sensor translation, lidar pitch is applied:

$$x_l = \cos \theta_l x_{\text{rel}} + \sin \theta_l z_{\text{rel}}, \quad (51)$$

$$y_l = y_{\text{rel}}, \quad (52)$$

$$z_l = -\sin \theta_l x_{\text{rel}} + \cos \theta_l z_{\text{rel}}. \quad (53)$$

The ideal planar measurement is

$$r = \sqrt{x_l^2 + y_l^2}, \quad \varphi = \text{atan2}(y_l, x_l). \quad (54)$$

The visibility gate is

$$x_l > 0, \quad r \leq r_{\max}, \quad |\varphi| \leq \frac{\Phi_{\text{ROI}}}{2}. \quad (55)$$

Optional non-deskewed motion distortion contributes

$$\sigma_{\varphi, \text{motion}} = \frac{|r_{\text{yaw}}| T_{\text{ROI}}}{\sqrt{12}}, \quad (56)$$

and total azimuth uncertainty is

$$\sigma_{\varphi} = \sqrt{\sigma_{\varphi, 0}^2 + \sigma_{\varphi, \text{motion}}^2}. \quad (57)$$

The observed polar coordinates are

$$r_{\text{obs}} = r + \sigma_r \eta_r, \quad \varphi_{\text{obs}} = \varphi + \sigma_{\varphi} \eta_{\varphi}, \quad \eta_r, \eta_{\varphi} \sim \mathcal{N}(0, 1). \quad (58)$$

They are converted back to Cartesian coordinates:

$$x_{\text{obs}} = r_{\text{obs}} \cos \varphi_{\text{obs}}, \quad y_{\text{obs}} = r_{\text{obs}} \sin \varphi_{\text{obs}}. \quad (59)$$

9 Fusion and perception errors

Fusion uses lidar position. Camera colour is attached only when the same cone passes the camera visibility gate. Lidar-only cones have unknown colour.

Optional dropout applies an independent Bernoulli trial:

$$D_i \sim \text{Bernoulli}(p_{\text{drop}}). \quad (60)$$

A detection is removed when $D_i = 1$. The number of optional false positives per frame is

$$N_{\text{false}} \sim \text{Poisson}(\lambda_{\text{false}}). \quad (61)$$

Their forward range and lateral position are sampled uniformly inside the configured region.

Camera and lidar execution latency are sampled as

$$t_{\text{exec}} = |\mathcal{N}(\mu_{\text{exec}}, \sigma_{\text{exec}}^2)|. \quad (62)$$

10 IMU

Accelerometer and gyroscope biases are independent random walks:

$$b_{k+1} = b_k + \sigma_{\text{rw}} \sqrt{\Delta t} \eta_k, \quad \eta_k \sim \mathcal{N}(0, 1). \quad (63)$$

Each channel uses a first-order bandwidth model:

$$\tau = \frac{1}{2\pi f_c}, \quad \alpha = \frac{\Delta t}{\tau + \Delta t}, \quad y_k^f = y_{k-1}^f + \alpha(y_k - y_{k-1}^f). \quad (64)$$

The sampled output is

$$y_k^m = y_k^f + b_k + \sigma_n \eta_k. \quad (65)$$

Accelerometer and gyroscope have independent acquisition clocks. Output messages contain the arithmetic mean of the configured number of buffered samples.

11 State estimator

For each estimated state component q ,

$$b_{q,k+1} = b_{q,k} + \sigma_{q,\text{rw}} \sqrt{\Delta t} \eta_{b,k}, \quad (66)$$

$$\hat{q}_k = q_k + b_{q,k} + \sigma_{q,n} \eta_{n,k}. \quad (67)$$

Position and yaw use one delayed queue; body velocities use another. The estimate is published only after the configured calibration interval and when both delayed samples are available.

12 Discrete timing

For device period T_d and physics step Δt ,

$$N_d = \max \left(1, \text{round} \frac{T_d}{\Delta t} \right). \quad (68)$$

A device runs when

$$k \bmod N_d = k_{\text{phase}}. \quad (69)$$

Independent randomized phases avoid artificial simultaneous sampling. Sampled values are held in caches and computation or transport latency is represented by queues indexed by their ready simulation step.

13 Implementation references

Model	Source
Body and wheel dynamics	<code>src_helpers/double_track.cpp</code>
Tyres	<code>src_helpers/tire_model.cpp</code>
Suspension	<code>src_helpers/suspension.cpp</code>
Steering	<code>src_helpers/steering_system.cpp</code>
Torque routing	<code>src_helpers/torque_allocation.cpp</code>
Camera, lidar, fusion	<code>src_helpers/cone_detector.cpp</code>
IMU	<code>src_helpers/imu_simulator.cpp</code>
Estimator and timing	<code>sim_loop.cpp</code>